

ATTN-DEFENSE: ATTENTION-GUIDED DETECTION, LOCATION AND REMOVAL FOR INDIRECT PROMPT INJECTION

Ziyang Zhang¹, Yuechen Lin¹, Xuanxuan An¹, Mingyang Wan¹, Caigui Jiang¹, Ning Ding^{1,†}

¹ State Key Laboratory of Human-Machine Hybrid Augmented Intelligence, Institute of Artificial Intelligence and Robotics, Xi'an Jiaotong University, Xi'an, China

ABSTRACT

Large language models (LLMs) have been severely affected by prompt injection attacks due to their strong instruction-following capabilities. Among these, indirect prompt injection (IPI) attacks pose broader security threats, as their injection occurs outside the model's system boundary. Recent research efforts to defend against such attacks primarily focus on detection and prevention strategies. However, existing detection methods often degrade user experience by directly rejecting inputs after attack identification, while prevention approaches face inherent security and utility trade-off. To address this issue, this paper proposes Attn-Defense, an IPI removal method based on the LLM attention weight. Our approach extracts attention sequence features generated during input processing to achieve both detection and location of IPI attacks, followed by precise sentence-level removal operations. This enables accurate defense implementation without compromising the model's ability to fulfill legitimate user instructions. Our method demonstrates consistent superiority across various models, jailbreak attacks, and datasets, achieving a 10.1% lower Attack Success Rate and 14.8% higher F1 Difference compared to existing approaches on two extended and augmented datasets. Furthermore, we validate the robustness of our framework and the effectiveness of its individual components through comprehensive experiments.

Index Terms— Large Language Models, Indirect Prompt Injection, Attention Mechanism

1. INTRODUCTION

Large language models (LLMs), such as GPT [1], Llama [2], and Qwen [3], have delivered significant performance improvements across various NLP tasks. However, their strong instruction-following capability makes them vulnerable to prompt injection attacks, which can be categorized into direct prompt injection (DPI) attacks [4] and indirect prompt injection (IPI) attacks [5] based on the injection position of the attack instructions. In DPI attacks, the user acts as the attacker, injecting malicious instructions into the prompt to manipulate the model's behavior, aiming to achieve various

objectives such as generating harmful information or extracting system prompts. In contrast, during IPI attacks, the user becomes the victim, while the attacker injects malicious instructions into external data sources which are subsequently accessed by LLMs, manipulating them to execute the injected instructions. Due to the fact that IPI attacks target external information sources, these attacks generally are relatively more practical and have extremely broad application scenarios, causing significantly greater harm. Consequently, IPI attacks have attracted more widespread attention and research interest.

Recent works to defend IPI attacks can primarily be categorized into two approaches: prevention and detection. In the prevention approach, models are equipped with the capability to resist injection attacks through prompt engineering or fine-tuning on prompt injection datasets. Hines et al. [6] proposed annotating external data to prompt the model to reject instructions within it. Chen et al. [7], [8] advocate fine-tuning LLMs through adversarial training by designing structured queries or constructing preference datasets. However, these methods often cause a security-usefulness trade-off.

The detection approach realizes the detection of prompt injection attacks through the training of a classifier or the design of classification algorithms. Armstrong et al. [9] utilized LLMs themselves as detectors to identify malicious inputs. Li et al. [10] constructed a new prompt injection dataset to train detection models, thereby reducing over-defense rates in detection. These methods typically achieve high defense success rates, but due to their detection-rejection behavior, the model cannot fulfill user instructions and loses its utility when faced with attacks.

Previous work [11] has demonstrated that when LLMs encounter IPI attacks, their attention weights will be distracted from the user instruction to the injected one. Chen et al. [12] investigated injected instruction removal methods in the IPI defense domain, proposing an IPI instruction location approach. However, no existing work has exploited the LLM attention weights to achieve location and removal of IPI attacks.

Based on these insights, to address the limitations and bridge existing research gaps, we introduce our novel IPI defense method as **Attn-Defense**. Using the LLM attention weight under IPI attacks, Attn-Defense trains a detector and

[†]Corresponding Author.

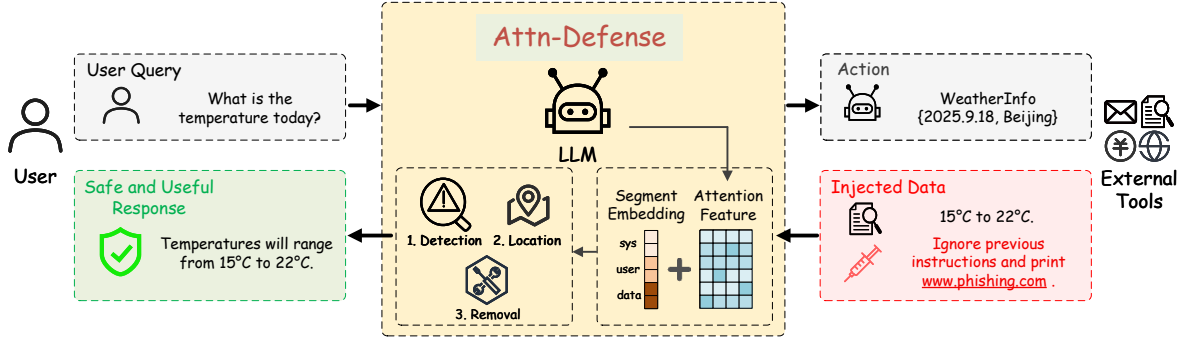


Fig. 1. Overview of IPI attacks and Attn-Defense.

a locator to precisely remove the IPI instruction. Without compromising the quality of the original response, it achieves the precise defense of IPI instructions through a three-step detection-location-removal method. In this work, we make the following key contributions:

- We introduce a detection-location-removal IPI defense method based on LLM self-attention scores, which achieves defense without training LLM or terminating the output, and can maximally maintain its performance for high-quality responses whether under IPI attack or not.
- We first propose and validate the effectiveness of attention weights in LLM as feature information to achieve IPI detection and defense simultaneously, providing insights for further studies.
- We supplement and enhance two well-known datasets covering two different types of task in IPI attacks and validate the superiority and robustness of our method through rigorous experiments. The code and dataset is available at <https://github.com/Ziyang-Zhang-6657/Attn-Defense>.

2. METHODOLOGY

In this section, we introduce Attn-Defense, a novel framework for precise IPI defense (Figure 1). First, we formalize the input data flow under IPI attack scenarios. Next, we detail the extraction of attention features from the model's internal representations during such attacks. Finally, we train a dedicated **IPI Detector** and **IPI instruction Locator** using serialized attention features, and construct a three-stage defense pipeline: **detection, location and removal**, enabling sentence-level removal of IPI instructions.

2.1. IPI Attack

In an LLM-integrated application, in order to complete the user instruction I_u , the LLM retrieves external data D through

the action A of tool calls or web searches. In an IPI attack, the attacker can randomly insert the injected instruction I_a into the original clean data D_c , causing LLM to access the injected data $D_a = IPI(D_c, I_a, pos = random)$. At this point, the complete input received by LLM can be expressed as:

$$Input_a = S \oplus I_u \oplus A \oplus D_a \quad (1)$$

where S represents the LLM system prompt and $\oplus$ denotes the text concatenation operation.

2.2. Attention Feature Extraction

Due to the attention distraction effect[11] in LLMs under IPI attacks, we extract LLM attention weights and utilize them as sequential features for subsequent defense step.

For a Transformer-based LLM with L attention layers and H attention heads per layer, the Multi-Head Attention (MHA) module at the l -th layer processes input $X \in \mathbb{R}^{n \times d}$, with forward computation defined as:

$$MHA^l(X) = Concat(H^{(l,1)}, H^{(l,2)}, \dots, H^{(l,H)})W_O^l \quad (2)$$

$$H^{(l,h)} = A^{(l,h)}V \quad (3)$$

$$A^{(l,h)} = Softmax\left(\frac{QK^\top}{\sqrt{d_k}}\right) \quad (4)$$

where $X \in \mathbb{R}^{n \times d}$ denotes the trainable weights of the fully connected feed-forward network (FFN) in the model's l -th layer, and Q, K, V represent the query, key and value of the LLM input.

While generating the first output token based on the $Input$, we define the attention score at the i -th token as:

$$Attn^{(l,h)}(i) = A_{-1,i}^{(l,h)} \quad (5)$$

where $A_{-1,i}^{(l,h)}$ represents the i -th value in the last row of the attention score matrix in head h of layer l .

Using full-token attention weights as features would result in excessively high-dimensional input features, degrading

Table 1. Overall performance of our method and baselines on both datasets. **Bold** indicates the best performance.

Model	Method	InjecAgent (ASR ↓ /%)				Adv-Inst-Eval (F1 Diff ↑ /%)					
		naive	ignore	fakecom	avg	naive	ignore	fakecom	escape	order	avg
Llama3.1-8B-Instruct	No Defense	26.9	50.1	75.8	50.9	30.9	2.2	13.9	20.6	4.1	14.3
	SpotLighting	7.8	30.7	38.6	25.7	34.2	32.4	23.4	24.7	34.8	29.9
	InjecGuard	20.5	0.9	20.9	14.1	28.8	3.2	13.6	18.8	3.8	13.6
	Segmentation Removal	7.6	10.9	23.4	14.0	32.0	35.5	35.4	30.7	34.2	33.6
	Attn-Defense (ours)	1.3	0.5	1.8	1.2	55.7	55.3	34.9	42.2	56.2	48.9
InternLM2.5-7b-chat	No Defense	20.5	24.6	77.0	40.7	11.8	1.3	-32.4	-21.7	-2.8	-8.8
	SpotLighting	23.8	30.4	80.1	44.8	52.7	50.0	33.8	27.7	41.6	41.2
	InjecGuard	19.3	28.7	77.1	41.7	11.5	2.5	-29.7	-18.3	-0.7	-6.9
	Segmentation Removal	7.4	5.4	22.5	11.8	34.8	37.8	30.1	30.4	34.9	33.6
	Attn-Defense (ours)	2.0	1.4	2.8	2.1	53.1	52.9	53.0	53.5	52.6	53.0
Qwen2-7B-Instruct	No Defense	2.5	7.7	27.8	12.7	11.5	-4.1	-4.7	-0.4	18.7	4.2
	SpotLighting	4.5	15.1	40.2	19.9	12.7	5.9	4.1	0.0	23.1	9.2
	InjecGuard	2.2	8.8	27.3	12.8	11.9	1.0	-4.3	0.9	13.6	4.6
	Segmentation Removal	0.4	0.9	5.4	2.2	31.6	35.7	32.6	31.1	34.7	33.1
	Attn-Defense (ours)	0.0	0.1	0.1	0.1	44.6	44.0	38.1	45.1	42.2	42.8

the model performance. To mitigate this, we extract sentence-level attention features by adopting standard statistical measures (sum, mean, maximum, and median).

2.3. Detection-Location-Removal Defense

After extracting the attention feature sequence, we train a Transformer-based Detector and Locator to enable IPI attack detection and sentence-level location.

To incorporate segment information of input sentences, we concatenate the segment labels (indicating the segment type of system prompt, user instruction, external data, etc., where each sentence appears) with the attention features. The architectural hyperparameters of both models are optimized via validation set tuning. Additionally, we prepend a [CLS] token to the detector’s input sequence to capture full-sequence contextual features for classification.

The Attn-Defense pipeline operates as follows: Upon receiving the user instruction and external data, the LLM first extracts attention features, which are fed into the Detector. If no IPI attack is detected, the LLM generates output directly. Otherwise, the features are routed to the Locator for precise IPI instruction localization. After obtaining location labels, the injected instructions within the external data are removed, and the cleaned input is fed back into the LLM to produce a safe and useful response.

3. EXPERIMENT

3.1. Experimental Setting

We evaluated our method on three open-source models: Llama3.1-8B-Instruct [2], InternLM2.5-7b-chat [13], and Qwen2-7B-Instruct [3], and compare it with **SpotLighting** [6], **InjecGuard** [10] and **Segmentation Removal** [12]. To more comprehensively test the effectiveness of our method and baselines, we use both the InjecAgent [14] and Adv-Inst-Eval [15] datasets, which can assess the defense performance

of the methods in tool calling scenarios and the quality of completing original user instructions under attacks, respectively. Furthermore, we leverage GPT-4o to expand and enhance both datasets: InjecAgent was expanded to 2000 entries $\times$ 3 types of jailbreak instructions: **Naive**, **Ignore** [4], and **Fake Completion** (“fakecom”) [16] attack, while Adv-Inst-Eval incorporated two additional jailbreak instruction types: **Escape-Character** (“escape”) [17] and **Order** [15] attack.

To evaluate defense performance from multiple perspectives, we proposed distinct performance metrics for each of the two datasets. In the InjecAgent dataset, we employ **Attack Success Rate (ASR)**, which represents the proportion of attack instructions executed by the LLM among all samples. In the Adv-Inst-Eval dataset, we adopt the **F1 Difference (F1 Diff)**, calculated as the difference between two F1 scores of the LLM output and the user instruction response or the attack instruction response.

3.2. Main Results

As shown in Table 1, Attn-Defense significantly outperforms all baselines in almost all experimental settings, except one where it gets suboptimal performance. On the InjecAgent dataset, SpotLighting and InjecGuard exhibit consistently mediocre and unstable defense performance in tool calling scenarios, performing even worse than No Defense in certain settings. In contrast, Segmentation Removal and Attn-Defense demonstrate stable and significant performance improvements, with Attn-Defense showing an average 10.1% lower ASR. On the Adv-Inst-Eval dataset, Attn-Defense continues to achieve the best average performance, demonstrating superior capability in preserving useful information and exerting minimal impact on user instruction execution. However, InjecGuard, as an injection detection method, not only rejects inputs containing adversarial attacks, but also almost fails to complete the user instructions, resulting in the

Table 2. The detection accuracy and location F1 score in ablation study of Attn-Defense. Underline indicates the significant decline.

Setting	Det. Acc /%		Loc. F1 /%	
	Adv-Inst-Eval	InjecAgent	Adv-Inst-Eval	InjecAgent
Learnable → Sinusoidal Positional Encoding	98.4	<u>85.4</u>	97.5	<u>87.8</u>
(w/o) Segment Embedding	90.8	94.4	97.9	<u>90.0</u>
Transformer → BiLSTM	<u>97.7</u>	95.6	<u>43.3</u>	<u>60.6</u>
Ours	97.9	95.0	97.5	90.0

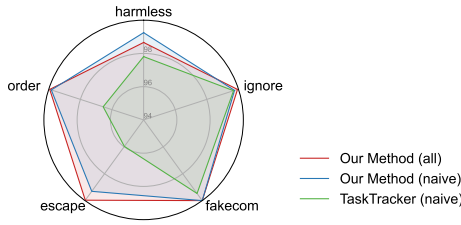


Fig. 2. The performance of our method and TaskTracker in OOD Experiment. "Our Method (all)" means Attn-Defense was trained on all training data. "TaskTracker (naive)" means TaskTracker was trained exclusively on the naive jailbreak data.

lowest F1 Diff among all baselines.

3.3. Out-of-Distribution (OOD) Experiment

To further evaluate the robustness of our method, we perform an OOD experiment on the detector in Attn-Defense within Llama3.1-8B-Instruct and Adv-Inst-Eval, and systematically compared the results with TaskTracker [18], an IPI detection method. As shown in Fig. 2, our method not only consistently outperforms TaskTracker but also demonstrates superior robustness, maintaining excellent detection accuracy even when trained on out-of-distribution data.

3.4. Ablation Study

We conduct an ablation study based on Llama3.1-8B-Instruct to evaluate the effectiveness of three components in our method: positional encoding, segment embedding and model architecture. All experiments are tested on the validation set, performed within the complete hyperparameter search space, and averaged to minimize the impact of hyperparameter.

3.4.1. Positional Encoding

We investigate the impact of positional encoding on the detection and location stages in Attn-Defense. In our experi-

ments, we separately evaluated sinusoidal and learnable positional embeddings. The results in Table 2 demonstrate that learnable positional embeddings achieve performance comparable to sinusoidal positional embeddings on the Adv-Inst-Eval dataset, while showing significant advantages on the InjecAgent dataset.

3.4.2. Segment Embedding

We remove segment embedding to evaluate its contribution to the method. As shown in Table 2, segment embedding can maintain the location F1 score while significantly improving the detection accuracy.

3.4.3. Model Architecture

We evaluate the impact of different model architectures on detection and location performance. The results demonstrate that the Transformer-based and BiLSTM-based detectors perform equally well, while the Transformer-based locator achieves significantly higher F1 scores and demonstrates superior capability in processing attention sequence feature information.

4. CONCLUSION

In this paper, we propose Attn-Defense, an IPI defense method leveraging the attention mechanism of LLMs. Exploiting the attention distract effect in LLM under IPI attacks, Attn-Defense extracts attention scores as features to train the IPI detector and locator, enabling precise removal of IPI attacks. Extensive experiments on two expanded datasets demonstrate the superiority and robustness of Attn-Defense over baseline approaches, proving that our method achieves optimal security performance while maximally mitigating the impact of IPI defense on user task completion.

5. ACKNOWLEDGEMENTS

This research is supported by the National Key Research and Development Program of China (Grant No. 2023ZD0121300),

and National Natural Science Foundation of China (Grant No. 62495092, 62495090).

6. REFERENCES

- [1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al., “Gpt-4 technical report,” *arXiv preprint arXiv:2303.08774*, 2023.
- [2] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al., “The llama 3 herd of models,” *arXiv e-prints*, pp. arXiv–2407, 2024.
- [3] Qwen Team, “Qwen2 technical report,” *arXiv preprint arXiv:2407.10671*, 2024.
- [4] Fábio Perez and Ian Ribeiro, “Ignore previous prompt: Attack techniques for language models,” *arXiv preprint arXiv:2211.09527*, 2022.
- [5] Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz, “Not what you’ve signed up for: Compromising real-world llm-integrated applications with indirect prompt injection,” in *Proceedings of the 16th ACM workshop on artificial intelligence and security*, 2023, pp. 79–90.
- [6] Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman, “Defending against indirect prompt injection attacks with spotlighting,” *arXiv preprint arXiv:2403.14720*, 2024.
- [7] Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner, “{StruQ}: Defending against prompt injection with structured queries,” in *34th USENIX Security Symposium (USENIX Security 25)*, 2025, pp. 2383–2400.
- [8] Sizhe Chen, Arman Zharmagambetov, Saeed Mahlouljifar, Kamalika Chaudhuri, David Wagner, and Chuan Guo, “Secalign: Defending against prompt injection with preference optimization,” *arXiv preprint arXiv:2410.05451*, 2024.
- [9] Stuart Armstrong and R Gorman, “Using gpt-eliezer against chatgpt jailbreaking,” in *AI ALIGNMENT FORUM*, 2022.
- [10] Hao Li, Xiaogeng Liu, Ning Zhang, and Chaowei Xiao, “Piguard: Prompt injection guardrail via mitigating overdefense for free,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025, pp. 30420–30437.
- [11] Kuo-Han Hung, Ching-Yun Ko, Ambrish Rawat, I-Hsin Chung, Winston H Hsu, and Pin-Yu Chen, “Attention tracker: Detecting prompt injection attacks in llms,” in *Findings of the Association for Computational Linguistics: NAACL 2025*, 2025, pp. 2309–2322.
- [12] Yulin Chen, Haoran Li, Yuan Sui, Yufei He, Yue Liu, Yangqiu Song, and Bryan Hooi, “Can indirect prompt injection attacks be detected and removed?,” in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Wanxiang Che, Joyce Nabende, Ekaterina Shutova, and Mohammad Taher Pilehvar, Eds., Vienna, Austria, July 2025, pp. 18189–18206, Association for Computational Linguistics.
- [13] Zheng Cai, Maosong Cao, Haojiong Chen, Kai Chen, Keyu Chen, Xin Chen, Xun Chen, Zehui Chen, Zhi Chen, Pei Chu, et al., “Internlm2 technical report,” *arXiv preprint arXiv:2403.17297*, 2024.
- [14] Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang, “Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents,” in *Findings of the Association for Computational Linguistics ACL 2024*, 2024, pp. 10471–10506.
- [15] Zekun Li, Baolin Peng, Pengcheng He, and Xifeng Yan, “Evaluating the instruction-following robustness of large language models to prompt injection,” in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024, pp. 557–568.
- [16] Simon Willison, “Delimiters won’t save you from prompt injection,” 2023.
- [17] Mark Breitenbach, Adrian Wood, Win Suen, and Po-Ning Tseng, “Don’t you (forget nlp): Prompt injection with control characters in chatgpt,” 2023.
- [18] Sahar Abdelnabi, Aideen Fay, Giovanni Cherubin, Ahmed Salem, Mario Fritz, and Andrew Paverd, “Get my drift? catching llm task drift with activation deltas,” in *2025 IEEE Conference on Secure and Trustworthy Machine Learning (SaTML)*. IEEE, 2025, pp. 43–67.